Hindsight 从入门到精通

> 版本与核实信息（核实日期：2026 年 10 月 26 日）
>
> - **教程基于版本线**：Hindsight 1.1（hindsight-api 最新稳定版 1.1.0，要求 Python ≥ 3.11；官方文档版本线 1.1）。文中命令与参数均与该版本线官方文档核对；不同版本行为差异处已单独标注。
> - **核实来源**：官方文档（hindsight.vectorize.io）、官方 GitHub 仓库（github.com/vectorize-io/hindsight）、PyPI 发布记录与官方变更日志。
> - **时效提示**：Hindsight 迭代较快，安装方式、配置项与 API 可能调整。写作中未能核实的细节一律省略或标注「以官方文档为准」；建议升级版本前后对照官方变更日志（Changelog）。

前言

本文面向希望为 AI 智能体（agent）构建长期记忆能力的开发者与工程团队，目标是把读者从「首次接触 Hindsight」带到「在生产环境中熟练运用」。

全文按能力成长划分为三篇：

篇章	能力目标	内容范围
第一篇 入门篇：会用	认识系统、完成部署、跑通三操作	第 1–3 章
第二篇 进阶篇：用好	深入数据模型、检索机制与各类集成	第 4–8 章
第三篇 精通篇：用精	生产部署、运维、安全与调优，最终落地综合案例	第 9–13 章

阅读约定：

1. 示例以 Python 客户端为主，涉及多客户端处给出表格对比；命令行示例默认在类 Unix shell 中执行，Windows 环境的写法在第 2.6 节单独说明。
2. 环境变量一律使用 `HINDSIGHT_API_` 前缀（服务端配置约定），配置层级在第 10 章展开。
3. 术语首次出现时给出定义，全文统一使用；术语表见附录。
4. 教程中的所有命令均已按上文核实日期与官方文档核对，可直接运行；凡属推断或建议的内容，文中会明确说明理由。

第一篇 入门篇：会用

第 1 章 AI 智能体为什么需要记忆

本章回答三个问题：大语言模型应用为什么需要外部记忆系统；Hindsight 在同类方案中的定位；以及使用它之前需要理解的基本概念。完成本章学习后，读者能够判断一款智能体应用是否需要记忆能力，并能准确说出 Hindsight 的四类记忆与三个核心操作。

1.1 上下文窗口的局限与记忆系统的定位

大语言模型通过上下文窗口接收输入，但这一窗口的容量有限，且每次调用都是一次独立推理：模型不会在两次会话之间自行保留任何信息。对持续服务的智能体而言，这带来三个明确问题：

1. **历史无法延续。**用户上周说明过的偏好、约束与任务背景，在新的会话中不会自动生效。
2. **信息无法沉淀。**多轮交互中产生的结论（例如「该用户偏好简洁回复」「该项目当前处于封版阶段」）缺少一个可积累、可修正的载体。
3. **检索不等于记忆。**用检索增强生成（RAG，Retrieval-Augmented Generation）把文档切片后按相似度召回，本质是「查询原文片段」，适合静态文档问答；它既不做事实抽取与去重，也不随新证据修正旧结论，用于承载「智能体记住了什么」并不合适。

记忆系统的职责因此可以概括为：把智能体与用户、环境交互产生的信息，加工为可检索、会演进的知识，并在恰当的时机提供给模型。

1.2 Hindsight 是什么：会学习的记忆系统

Hindsight 是 Vectorize 公司开源（MIT 许可）的智能体记忆系统，定位是「会学习的智能体记忆」（agent memory that learns）。它与侧重会话历史回放的系统不同，重点在于让智能体在使用过程中积累、修正知识。截至本教程核实时，项目在 GitHub 上约 53.8k star，在长对话记忆基准 LongMemEval 上取得了领先成绩，该成绩由弗吉尼亚理工大学 Sanghani 智能与数据分析中心与 The Washington Post 独立复现；实时榜单见 benchmarks.hindsight.vectorize.io，方法论文见 arXiv 2512.12818。

形态上，Hindsight 是一个独立服务：内置 HTTP API、Web 控制台（Control Plane）、后台任务进程（worker）、CLI 工具与多种语言的客户端 SDK。智能体通过 retain（写入）、recall（检索）、reflect（综合推理）三个操作与它交互，后端由 PostgreSQL 与向量扩展支撑持久化。

1.3 四类记忆与三大操作

Hindsight 将记忆组织为四类，识别各自的含义是后续所有章节的基础：

记忆类型	英文原文	含义	来源
世界事实	world fact	关于世界的客观事实，如「炉子会发热」	写入时由模型自动抽取
经历	experience	智能体自身经历过的事件与对话，如「触碰过炉子并被烫伤」	写入时由模型自动抽取
观察	observation	由多条记忆整合生成的、带证据支撑的信念，可被新证据强化、弱化或修正	后台整合自动生成
心智模型	mental model	针对某个问题预先写好的「标准答案」，随知识演进在后台自动改写	用户定义问题后生成

三个核心操作构成与记忆系统交互的完整闭环：

- **retain（写入）**：提交原始内容（对话、文档、笔记），系统用 LLM 抽取事实、实体与时序信息，建立检索所需的结构化索引。
- **recall（检索）**：按自然语言查询返回相关记忆，内部并行执行语义、关键词、图结构、时间四种检索策略并融合排序。
- **reflect（综合推理）**：在检索的基础上做多步分析和整合，回答需要思考而非简单查找的问题。

四类记忆与三个操作的关系可以概括为：retain 产生世界事实与经历；后台机制把事实加工为观察；用户定义的心智模型基于观察与事实生成；recall 与 reflect 则消费这一切。

1.4 与 RAG 的对比与适用判断

Hindsight 与传统 RAG 解决的是相邻但不同的问题，官方也提供了专门的对比文档（rag-vs-hindsight）。两者差异的主要方面如下（对比是客观描述，选型建议附后）：

维度	传统 RAG（切片 + 向量检索）	Hindsight
存储对象	原文切片	抽取后的结构化事实、实体关系与时序
时间处理	通常无	事实带发生时间与提及时间，支持时间窗检索
知识演进	文档更新靠重建索引	观察随新证据修正，心智模型自动改写
检索方式	单一向量相似度	语义、关键词、图、时间四路并行 + 融合重排
典型用途	静态文档问答	会话型与任务型智能体的长期记忆

选型建议：若应用是围绕固定文档库的问答（如产品手册检索），RAG 已足够；若智能体需要记住用户与自身的历史、随交互持续修正认知（如个人助理、客服坐席、项目协作智能体），Hindsight 更合适。官方亦承认：对简单的工作流编排场景（如 n8n 配置式流程），引入 Hindsight 可能过重。

1.5 本教程的环境设定

后续章节的默认设定为：Linux x86_64 或 Windows x86_64 开发机、Python 3.11 及以上、Docker Desktop（第 2 章起使用）、可用的 LLM API 密钥（默认以 OpenAI 或 DeepSeek 为例，配置项对所有主流提供商通用）。macOS 用户（含 Apple Silicon 与 Intel 机型）的差异在第 2.7 节说明。

第 2 章 安装部署：Docker / pip / 内嵌三种形态与平台差异

安装阶段最常见的困难是「为一个演示跑了一整套生产依赖」，或者反过来「用了开发配置却把它带上了生产」。本章给出三种部署形态的边界：Docker 一键启动适合快速验证；pip 安装适合把服务作为主机上的常驻进程管理；内嵌模式适合在应用进程内直接运行。完成本章学习后，读者能够针对自己的操作系统与场景选定一种形态并完成部署。

2.1 部署形态总览与选型

形态	安装物	适用场景	特点
Docker（全量镜像 or 精简镜像）	`ghcr.io/vectorize-io/hindsight:<tag>`	快速上手、开发、中小规模部署	一条命令，含内嵌数据库，开箱即用
pip 常驻服务	`hindsight-api`（或 `hindsight-api-slim`）	主机常驻、由 systemd 等管理	灵活控制端口与进程参数
Python 内嵌	`hindsight-all`（或 `hindsight-all-slim`）	单机应用内直接运行，无外部服务器	进程内启动或后台守护进程
Kubernetes / 云托管	Helm chart 或 Hindsight Cloud	生产环境	第 9 章展开

三种形态共用同一套 HTTP API 与客户端 SDK；客户端代码不因服务端部署形态而改变。

2.2 前置条件：LLM 提供商与资源要求

Hindsight 的工作流（事实抽取、实体消歧、答案生成）依赖外部 LLM。要点如下：

1. **结构化输出**：选定模型需支持结构化输出（structured output）。官方快速上手页将 Groq 与 `gpt-oss-20b` 列为兼顾速度与成本的推荐组合；示例中常用 `gpt-4o-mini`、`gpt-5-mini` 等，行为一致。
2. **提供商范围**：通过环境变量 `HINDSIGHT_API_LLM_PROVIDER` 配置，支持 25 种以上提供商，包括 `openai`、`anthropic`、`gemini`、`groq`、`bedrock`、`vertexai`、`minimax`、`deepseek`、`ollama`、`lmstudio` 等；支持任意 OpenAI 兼容端点（配合 `HINDSIGHT_API_LLM_BASE_URL`）；也可通过 `openai-codex`、`claude-code`、`cursor`、`github-copilot` 直接使用现有订阅；网关类 `litellm`、`litellmrouter` 可覆盖其余提供商。
3. **密钥环境变量**：`HINDSIGHT_API_LLM_API_KEY`（按提供商传入对应密钥）。
4. **嵌入与重排**：全量包内置本地嵌入模型（BGE，约 130 MB）与交叉编码器重排模型（MiniLM，约 90 MB），无需额外配置；精简包（slim）则要求配置外部嵌入与重排服务，或使用内置 ONNX 后端（见 2.10 节）。

资源占用参考（来自官方安装文档）：全量 API 进程最低内存 1.5 GB（推荐 2 GB）；精简形态 512 MB（推荐 1 GB）；Web 控制台 128 MB；PostgreSQL 512 MB 起。CPU 两个逻辑核可支撑开发与基本负载；生产流量下本地重排模型建议配备 GPU 或改用外部重排服务。

2.3 Docker 部署（推荐入门方式）

一条命令即可启动含 API、控制台与内嵌 PostgreSQL 的完整容器（Linux、macOS、Windows 均适用；命令与官方 README 一致）：

```
export OPENAI_API_KEY=sk-xxx

docker run -it --pull always --name hindsight --restart unless-stopped -p 8888:8888 -p 9999:9999 \
  -e HINDSIGHT_API_LLM_API_KEY=$OPENAI_API_KEY \
  -v hindsight-data:/home/hindsight/.pg0 \
  ghcr.io/vectorize-io/hindsight:latest
```

参数说明：

- `-p 8888:8888`：Hindsight HTTP API 端口。
- `-p 9999:9999`：Control Plane Web 控制台端口。
- `-e HINDSIGHT_API_LLM_API_KEY=...`：将 LLM 密钥传入服务。若使用其他提供商，再补 `HINDSIGHT_API_LLM_PROVIDER` 与 `HINDSIGHT_API_LLM_MODEL`（例如 DeepSeek）。
- `-v hindsight-data:/home/hindsight/.pg0`：命名卷，持久化内嵌 PostgreSQL 数据。推荐使用命名卷而非主机目录绑定：容器以非 root 用户（UID 1000）运行，若绑定主机目录，目录属主必须是 UID 1000，否则内嵌数据库会因权限拒绝启动失败。
- `--restart unless-stopped`：随 Docker 守护进程自动重启。

启动后访问确认：API 健康页 http://localhost:8888/docs（OpenAPI 文档），控制台 http://localhost:9999。

两个生产相关注意事项（细节在第 9、10 章展开）：

1. 生产环境建议为容器设置稳定的 `HINDSIGHT_API_WORKER_ID`（例如 `-e HINDSIGHT_API_WORKER_ID=hindsight-prod`）。worker 以容器主机名作为身份，默认主机名是容器 ID，每次重建都会变化，导致中断时正在处理的任务无法被新实例认领。
2. 镜像提供两种变体：全量（`latest`，内含本地嵌入与重排模型，AMD64 约 9 GB / ARM64 约 3.7 GB）与精简（`latest-slim`，约 500 MB，要求外部嵌入与重排服务）。具体变体标签见 2.7 节。

2.4 pip 安装运行

适合将服务作为主机进程管理的场景。要求 Python 3.11 及以上。

```
pip install hindsight-api
export HINDSIGHT_API_LLM_API_KEY=sk-xxx

hindsight-api
```

默认使用内嵌 PostgreSQL（pg0），数据目录为 `~/.hindsight/data/`，启动后监听 http://localhost:8888。常用的启动参数（均可选）：

```
hindsight-api --port 9000          # 自定义端口，默认 8888
hindsight-api --host 127.0.0.1     # 仅绑定本机回环地址
hindsight-api --workers 4          # 多进程并发处理
hindsight-api --log-level debug    # 调试日志
```

连接自建 PostgreSQL 的方式（须已创建数据库并启用向量扩展，见 9.5 节）：

```
export HINDSIGHT_API_DATABASE_URL=postgresql://user:pass@localhost:5432/hindsight
export HINDSIGHT_API_LLM_API_KEY=sk-xxx
hindsight-api
```

另有精简包 `hindsight-api-slim`（不含本地模型，体积小，要求配置外部嵌入与重排服务，生产部署常用）。注意 macOS Intel 机型上应使用精简包：全量包依赖的本地 ML 模型未发布 Intel Mac 版本，直接安装会回退到数月前的旧版本。

2.5 内嵌模式：hindsight-all

当 Python 应用希望「不部署独立服务」时，使用 `hindsight-all`：

```
pip install hindsight-all
```

提供两种内嵌方式：

方式一：进程内服务器（HindsightServer），服务器运行在应用的后台线程，适合单进程应用：

```python
import os
from hindsight import HindsightServer, HindsightClient

with HindsightServer(
    llm_provider="openai",
    llm_model="gpt-5-mini",
    llm_api_key=os.environ["OPENAI_API_KEY"],
) as server:
    client = HindsightClient(base_url=server.url)
    client.retain(bank_id="my-bank", content="Alice works at Google")
    results = client.recall(bank_id="my-bank", query="Where does Alice work?")
```

方式二：后台守护进程（HindsightEmbedded），服务器作为常驻守护进程运行，可被多个 Python 进程或会话共享，首次使用时自动启动：

```python
import os
from hindsight import HindsightEmbedded

client = HindsightEmbedded(
    llm_provider="openai",
    llm_api_key=os.environ["OPENAI_API_KEY"],
)
client.retain(bank_id="alice", content="Alice prefers concise answers.")
results = client.recall(bank_id="alice", query="How should I respond to Alice?")
```

内嵌包同样基于内嵌 PostgreSQL（pg0）。官方还提供 Node.js 内嵌包 `hindsight-all-npm` 与配套的守护进程 CLI（用法见官方文档 sdks 章节的 embed 页面）。Intel Mac 上改用 `hindsight-all-slim`。

2.6 Windows 与国内网络环境部署

Windows 原生部署：`hindsight-api` 在 Windows 上支持内嵌数据库开箱即用。在命令提示符（cmd）中：

```
pip install hindsight-api

set HINDSIGHT_API_LLM_PROVIDER=openai
set HINDSIGHT_API_LLM_API_KEY=sk-xxx
set HINDSIGHT_API_LLM_MODEL=gpt-4o-mini

hindsight-api
```

PowerShell 中设置环境变量使用 `$env:HINDSIGHT_API_LLM_PROVIDER = "openai"` 等写法，含义相同。Windows 也可使用外部 PostgreSQL（`winget install PostgreSQL.PostgreSQL.17` 后按官方安装文档「Windows」一节编译 pgvector 并启用 `CREATE EXTENSION vector;`），细节以官方文档为准。

受限网络环境（官方安装文档专门给出）：以下配置组合经官方文档验证可用，要点为：LLM 使用 DeepSeek，嵌入与重排走本地模型，并为 Hugging Face 工具链设置国内镜像站点：

```
set HF_ENDPOINT=https://hf-mirror.com

set HINDSIGHT_API_LLM_PROVIDER=deepseek
set HINDSIGHT_API_LLM_API_KEY=sk-your-deepseek-key
set HINDSIGHT_API_LLM_MODEL=deepseek-v4-flash
set HINDSIGHT_API_LLM_BASE_URL=https://api.deepseek.com

set HINDSIGHT_API_EMBEDDINGS_PROVIDER=local
set HINDSIGHT_API_EMBEDDINGS_LOCAL_MODEL=BAAI/bge-small-en-v1.5

set HINDSIGHT_API_RERANKER_PROVIDER=flashrank

hindsight-api
```

说明：`HF_ENDPOINT` 是 Hugging Face 工具链（huggingface_hub）的环境变量，用于本地模型下载加速，不是 Hindsight 自身解析的配置项。

2.7 平台支持差异与镜像标签

官方支持矩阵（安装文档 2026 年 10 月版）：

平台	Docker	pip 常驻	内嵌 pg0
Linux（x86_64、ARM64）	完整支持	完整支持	完整支持
macOS（Apple Silicon / arm64）	完整支持	完整支持	完整支持
macOS（Intel / x86_64）	完整支持	仅 slim 包	完整支持
Windows（x86_64）	完整支持	完整支持	完整支持

Docker 镜像可以按需选择以下标签（API 与控制台合一镜像为例）：

```
ghcr.io/vectorize-io/hindsight:latest          # 全量，最新版
ghcr.io/vectorize-io/hindsight:latest-slim     # 精简，最新版
ghcr.io/vectorize-io/hindsight:0.4.9           # 全量，指定版本（示例）
ghcr.io/vectorize-io/hindsight:<版本号>-slim   # 精简，指定版本
ghcr.io/vectorize-io/hindsight-api:latest      # 仅 API
ghcr.io/vectorize-io/hindsight-control-plane:latest  # 仅控制台
```

已发布镜像使用 Cosign 签名，可按官方文档提供的方法校验（可选步骤）。

2.8 连接控制台 Web UI

控制台（Control Plane）已在 Docker 镜像中内置（端口 9999）。pip 部署时默认不启动控制台，可用 npx 单独拉起（要求 Node.js）：

```
npx @vectorize-io/hindsight-control-plane --api-url http://localhost:8888
```

常用选项：

命令行选项	环境变量	默认值	说明
`-p, --port`	`PORT`	9999	监听端口
`-H, --hostname`	`HOSTNAME`	0.0.0.0	绑定地址
`-a, --api-url`	`HINDSIGHT_CP_DATAPLANE_API_URL`	http://localhost:8888	API 服务地址
—	`HINDSIGHT_CP_ACCESS_KEY`	未设置	设置后打开控制台需输入访问密钥

控制台提供记忆库管理、实体与知识图谱浏览、recall / reflect 交互式测试、操作历史查看等功能。CLI 亦有等价命令 `hindsight ui`（同样经 npx 启动）。

2.9 服务端可用的 LLM 提供商速览

配置任何提供商均只需三件套：`HINDSIGHT_API_LLM_PROVIDER`（提供商名）、`HINDSIGHT_API_LLM_API_KEY`（密钥）、按需 `HINDSIGHT_API_LLM_MODEL`（模型名）与 `HINDSIGHT_API_LLM_BASE_URL`（自建或兼容端点）。全部支持的名称以官方文档 models 页为准，本教程示例在 `openai`、`deepseek`、`groq` 之间取材。

2.10 精简包与本地模型

对部署体积与内存占用敏感的场景使用精简包：`hindsight-api-slim`（服务）或 `hindsight-all-slim`（内嵌）。精简包不含本地嵌入与重排模型，须二选一：

1. 配置外部提供商（如 OpenAI、Cohere、TEI、NVIDIA NIM 等）承担嵌入与重排；
2. 使用精简包附带的进程内 ONNX 后端（安装 `hindsight-api-slim[local-onnx]`）。

选择理由：精简包镜像约为 500 MB，稳态内存约 512 MB，明显小于全量包；代价是引入对外部服务的依赖。第 12 章会再次从成本视角讨论该取舍。

第 3 章 第一个记忆智能体：retain、recall、reflect 全流程

本章把「记入 — 取回 — 作答」三条主命令各跑通一遍，并让读者对后台处理有真实感受。全部示例基于第 2 章任意一种已启动的服务端（地址 `http://localhost:8888`）。

3.1 安装客户端并确认连通

官方 Python 客户端（对应请求在 http://localhost:8888 的 API 服务）：

```
pip install hindsight-client
```

```
from hindsight_client import Hindsight

client = Hindsight(
    base_url="http://localhost:8888",
    timeout=30.0,          # 请求超时秒数，可省略
    # api_key="...",       # 服务端启用鉴权时才需要
)

version = client.get_version()
print(version.api_version)   # 应输出与部署版本一致的 API 版本号
```

多语言客户端对照：

客户端	安装方式	客户端类
Python	`pip install hindsight-client`	`Hindsight`
Node.js / TypeScript	`npm install @vectorize-io/hindsight-client`	`HindsightClient`
Go	`go get github.com/vectorize-io/hindsight/hindsight-clients/go`	官方生成的 Go API
CLI	`curl -fsSL https://hindsight.vectorize.io/get-cli | bash`	—

（Node.js 基本用法与 Python 等价：`const { HindsightClient } = require('@vectorize-io/hindsight-client');` 创建实例后调用 `await client.retain('my-bank', '...')` 等。）

3.2 retain：写入第一条记忆

```
client.retain(
    bank_id="my-bank",
    content="Alice works at Google as a software engineer",
)
```

参数含义：

- `bank_id`：记忆库标识。**记忆库（bank）是隔离的记忆存储单元**，可以理解为「一个智能体或一个用户的一个大脑」；库之间严格隔离，不会互相串数据。bank 命名不可使用下划线开头的保留前缀（保留前缀规则见第 5.1 节）。
- `content`：待写入的原始内容，是唯一必填项。

执行后系统并非原样存下这句话，而是：把内容切片，调用 LLM 抽取事实（此处会得到「Alice 任职于 Google」「Alice 是软件工程师」级别的事实）、识别实体（Alice、Google）、记录时间线，并建立供四种检索策略使用的索引。原始文本默认不逐字入库（可配合 6.4 节的 chunks 选项取回来源片段）。

同步调用还会返回 token 用量与处理计数（响应字段 `usage`、`items_count` 等）；异步用法见第 4 章。

写入对话的推荐格式：保留说话人与时间，便于模型把事实归属到正确的人并解析相对时间表达：

```
conversation = "\n".join([
    "Alice (2024-03-15T09:00:00Z): Hi Bob! Did you end up going to the doctor last week?",
    "Bob (2024-03-15T09:01:00Z): Yes, finally. Turns out I have a mild peanut allergy.",
])

client.retain(
    bank_id="my-bank",
    content=conversation,
    context="team chat",                    # 来源场景标签，会注入抽取提示词
    timestamp="2024-03-15T09:04:00Z",       # 内容发生时间（ISO 8601）
    document_id="chat-2024-03-15-alice-bob", # 逻辑文档 ID，实现幂等重写
)
```

三个参数的解释（此处为首次出现，后文沿用）：

- `context`：描述内容来源或情境的短标签（如 `team chat`、`support ticket`）。同一句话在不同 context 下抽取结论可能不同，官方将持续、一致地提供 context 列为提升记忆质量最有效的做法之一。
- `timestamp`：内容所述事件的发生时间。缺省时取当前时间；传入 ISO 8601 字符串后模型以它为锚点解析「上周」「下季度」等相对时间；传 `"unset"` 表示内容没有现实时间（如参考资料、小说文本）。
- `document_id`：调用方自定义的文档标识。同一 bank 中重复提交同一 `document_id` 时，旧文档及其全部记忆会被删除并重新处理，因此对持续增长的内容（如聊天串流）可以安全地重发全量内容而不会累积重复记忆。

3.3 recall：检索记忆

```
results = client.recall(bank_id="my-bank", query="What does Alice do?")

for r in results.results:
    print(f"{r.text} (type: {r.type})")
```

预期输出形如（实际排序与内容取决于已写入的记忆）：

```
Alice works at Google as a software engineer (type: world)
```

`results.results` 中每条结果包含 `text`（事实文本）、`type`（`world` / `experience` / `observation`）、`context`、`metadata`、`tags`、`entities`（关联实体名）、`occurred_start` / `occurred_end`（事件发生时间）、`mentioned_at`（写入时间）、`document_id` 等字段。时间类查询（如 `What happened in June?`）依赖写入时提供的时间信息，这正是第 3.2 节强调 timestamp 的原因。

3.4 reflect：生成回答

```python
answer = client.reflect(
    bank_id="my-bank",
    query="Tell me about Alice",
    context="preparing for a meeting",
)
print(answer.text)
```

`reflect` 与 `recall` 的差别：recall 返回事实列表，由调用方自己组织进提示词；reflect 则由 Hindsight 在服务端完成检索、多步推理与答案生成，并按 bank 的 disposition（人格化推理倾向，第 5 章定义）调整语气，返回 `answer.text` 直接可用。生成过程会优先使用心智模型，其次观察，最后原始事实（检索阶梯见第 7 章）。`budget` 参数控制推理投入（`low` / `mid` / `high`，第 6 章详解），复杂问题可显式调高。

3.5 CLI 与多语言客户端速览

CLI 适合脚本调用、运维检查与快速验证。安装后先配置服务地址：

```bash
hindsight configure --api-url http://localhost:8888
# 或使用环境变量（优先级最高）：
export HINDSIGHT_API_URL=http://localhost:8888
export HINDSIGHT_API_KEY=your-api-key
```

对应本章三条主命令的 CLI 写法：

```bash
# 写入
hindsight memory retain my-bank "Alice works at Google as a software engineer"
hindsight memory retain my-bank "Bob loves hiking" --context "hobby discussion"

# 检索（budget 控制深度；-o 控制输出格式）
hindsight memory recall my-bank "What does Alice do?" --budget high -o json

# 综合推理
hindsight memory reflect my-bank "What do you know about Alice?"
```

另有两个常用交互入口：`hindsight ui` 打开 Web 控制台；`hindsight explore` 打开终端交互式浏览器（浏览 bank、搜索记忆、查看实体与文档，按 `q` 退出）。

3.6 观察后台处理：记忆如何「生长」

首次写入后立刻 recall，返回可能只有原始事实。这是正常的：观察（observation）与心智模型由后台任务异步生成，需要数秒到数分钟不定。可在控制台的 bank 页面查看这些派生知识的生成结果，或在 CLI 中：

```bash
hindsight bank stats my-bank        # 查看库内记忆构成与统计
hindsight memory recall my-bank "What have we learned about Alice?" \
  --fact-type observation           # 仅检索观察类型
```

（`--fact-type` 是 recall 的 CLI 参数，用于按类型过滤；SDK 对应参数为 `types`。）

本章小结：读者至此完成了写入、检索、生成三步闭环，并理解了 bank、context、timestamp、document_id 四个基础参数。下一章起引入数据模型与管理机制。

第二篇 进阶篇：用好

第 4 章 记忆类型与数据模型：world / experience / observation / mental model

入门篇的示例把记忆系统当作一个黑盒使用；本章打开这个盒子，说明写入的内容如何被加工，四类记忆分别承载什么，以及文档、标签与多语言在数据模型中的位置。完成本章学习后，读者能够为具体数据选择正确的写入方式，并能解释 recall 返回结果中各字段的来源。

4.1 世界事实与经历

每条通过 retain 提交的内容，都会被归入两条通道之一：以第三人称客观陈述为主的，形成世界事实（world fact）；以智能体自身视角记录交互与事件的，形成经历（experience）。这一分类由模型在抽取阶段完成，两条通道随后都会被表示为实体、关系、时序的组合，并配备稀疏与稠密向量索引以支撑后续检索。

区分两者的价值在于查询意图不同：客观知识查询（「在哪儿上班」）命中 world 类型为主，经历类查询（「我们推荐过什么」）命中 experience 类型为主。recall 支持按类型过滤，检索成本随类型收窄而降低（第 6 章）。

4.2 观察（observation）：整合后的信念

观察是 Hindsight 区别于普通向量数据库的关键数据结构。其要点如下：

1. **自动生成**：retain 之后，后台任务（consolidation，整合）把相关的事实簇去重、合并，生成观察。观察不是原文的摘要，而是该记忆库沉淀下来的一条结论性认知，例如「用户对会议时长敏感，倾向 30 分钟以内的短会」。
2. **带证据链**：每条观察保留支撑它的记忆，且引用原文，并记录证据计数（同一条观察被多少证据支撑过）。
3. **精炼而非覆盖**：新证据到达时，观察被强化、弱化或扩展，而不是被新条目静默替换。
4. **参与检索**：recall 的类型过滤支持 `observation`；reflect 的检索阶梯将观察列在原始事实之上（第 7 章）。

与观察相关的管理命令（均已核实）：

```
hindsight memory history my-bank "$MEMORY_ID"             # 查看某条记忆单元的观察演进历史
hindsight memory clear-observations my-bank "$MEMORY_ID" # 清除该记忆单元的观察（保留核心事实）
hindsight bank clear-observations my-bank -y              # 清除整个 bank 的观察
hindsight bank consolidation-recover my-bank              # 整合卡住或失败时尝试恢复
```

观察作用域（observation scopes）：当同一 bank 服务多个主题或多方角色时会用到。retain 的 `observation_scopes` 参数控制该条记忆参与哪些观察的整合，取值包括：

取值	行为
`combined`（默认）	以该记忆的全部标签为一个作用域，产出一条完整标签集的观察
`shared`	所有记忆汇入一个无标签的全局观察池，适合按会话随机打标的场景
`per_tag`	每个标签各自产出一条独立观察，适合多方对话（如同时按学生、教师归档）
`all_combinations`	标签的所有子集各产一条观察，规模随标签数增长较快
自定义列表	显式给出若干标签集合（支持 `topic:*` 通配），只产指定作用域

未显式传入时，沿用 bank 配置 `default_observation_scopes`（默认 `combined`）。作用域之间的整合使用严格的全匹配，保证一个作用域的观察不会混入另一作用域；已入库文档可经文档 PATCH 调整作用域。初学阶段保持默认即可，仅在「同一 bank 需要按标签独立沉淀结论」时使用。

4.3 retain 的底层加工：事实抽取、实体与时序

retain 同步调用背后发生四件事，理解它们有助于写出更高质量的输入：

1. **切片与抽取**：内容被切片后逐片交给 LLM 抽取结构化事实。响应中的 token 用量（`usage`）记录的就是这些模型调用。
2. **实体识别与消歧**：自动识别实体并在全 bank 范围内解析名字变体（例如「Dr. Waller」与「Dr Waller」归并为同一实体，依据名称相似度与共现强度）。若调用方确定某些实体不可被抽取遗漏，可在请求中传 `entities=[{"text": "张伟", "type": "PERSON"}]` 显式注入；`resolve_entities=False` 则关闭对调用方传入名称的自动合并且严格按原文建档（自动抽取的实体始终会做消歧）。
3. **时序解析**：为每条事实记录事件发生时间（`occurred_start` / `occurred_end`）与写入时间（`mentioned_at`），二者分离。这正是时间检索与续写对话式更新的基础。
4. **规范化与索引**：抽取结果转换为规范实体、时序与稀疏、稠密向量索引。

幂等与增量：`document_id` 相同的重写（默认 `update_mode="replace"`，删除旧文档后全量重建）；持续增长的内容可改用 `update_mode="append"`，仅新切片触发抽取，省去重复处理。

批量与文件：

```python
client.retain_batch(
    bank_id="my-bank",
    items=[
        {"content": "Alice works at Google", "context": "career", "document_id": "conv_msg_1"},
        {"content": "Bob is a data scientist", "context": "career", "document_id": "conv_msg_2"},
    ],
   retain_async=False,
)

result = client.retain_files(
    bank_id="my-bank",
    files=["quarterly-report.pdf"],          # 路径，客户端负责读取上传
    context="quarterly report",
)
print(result.operation_ids)                  # 文件处理固定异步，逐文件返回 operation_id
```

文件支持范围（官方 API 文档）：PDF、DOCX / DOC、PPTX / PPT、XLSX / XLS、图片（OCR 能力取决于配置的解析器）、音频（转录）、HTML 与常见纯文本格式；单次最多 10 个文件、合计 100 MB。上传的原始文件由服务端保存（默认存 PostgreSQL，生产可经 `HINDSIGHT_API_FILE_STORAGE_TYPE` 配置为 S3、GCS、Azure）。

异步与安全重试：`retain_async=True`（或 CLI 的 `--async`）立即返回 operation_id，由后台 worker 处理。为避免响应丢失后盲目重试造成重复抽取，可自带 `operation_id`（UUID）重发：同 ID 重提返回原操作、不新增任务；ID 已被其他操作占用则报 HTTP 409。

降低后台抽取成本：异步 retain 可开启提供商批量接口（Provider Batch API），将抽取任务以批量作业方式提交给 OpenAI、Groq 或 Gemini，换取最多 24 小时的处理窗口，成本约降 50%：`export HINDSIGHT_API_RETAIN_BATCH_ENABLED=true`。

4.4 文档、标签与多语言

文档（document）：一次带 `document_id` 的 retain 请求产出一个文档；无 `document_id` 的多个条目合并为一个随机 ID 文档，部分条目带 ID 时未带 ID 的条目各自独立成档。文档是删除、重建、打标签的管理单位。CLI 可直接管理：

```bash
hindsight document list my-bank
hindsight document get my-bank "$DOCUMENT_ID"
hindsight document update my-bank "$DOCUMENT_ID" --tags project-x,meetings
hindsight document delete my-bank "$DOCUMENT_ID"      # 删除文档及其全部记忆
```

标签（tags）：决定记忆的可见性范围。写入时给的标签在 recall 时按匹配规则过滤，典型约定为 `user:<id>`、`session:<id>`、`room:<id>`、`topic:<name>`。bank 提供 list-tags 端点返回现有标签与计数（供界面补全）。详细过滤语义在第 6.3 节展开。

元数据（metadata）：任意字符串键值对，随记忆存储并在每次 recall 结果中返回，供调用方做客户端过滤或回链（如引用来源 URL、工单号）。

多语言：输入语言在端到端被检测并保留，实体保持原文字形（「张伟」不会被转写为「Zhang Wei」），事实保持原始语言。无需为中文场景做额外配置；检索查询同样中英皆可。

第 5 章 记忆库管理：隔离、使命、disposition 与 bank 模板

同一套部署往往要同时服务许多用户、许多智能体或许多项目。Hindsight 的答案是记忆库（bank）：每库一个「大脑」，互相隔离。本章讲清 bank 的机制、个性化配置，以及用模板批量建档的做法。完成本章学习后，读者能够为多租户、多智能体场景设计出合理的库划分方案。

5.1 记忆库的隔离机制

bank 是隔离的记忆存储单元，隔离是严格的：跨库不泄漏，不同 bank 之间没有共享数据。三库以下的划分常按下列三类身份进行：

划分方式	典型命名	适用场景
按最终用户	`user-alice`、`user-123`	个人助理类：每位用户一个库
按智能体	`support-agent`、`pm-agent`	按职能分工各有记忆的智能体
按项目	`repo-myproject`	项目协作或代码库记忆（第 8.5 节即此用法）

库的标识由调用方指定并按需即时创建（首次 retain 一个新 `bank_id` 即自动建库）。命名注意：不可使用非法 ID 与下划线开头的保留前缀（1.1.0 起在导入等操作中会直接拒绝；具体字符规则以官方 API 文档为准）。

5.2 创建与配置：mission 与 disposition

显式创建 bank 时可一并给出两份「人格说明」：

```
client.create_bank(
    bank_id="assistant-prod",
    name="Assistant",
    mission="You're a helpful AI assistant - keep track of user preferences and conversation history.",
    disposition={
        "skepticism": 3,    # 1-5: 轻信到怀疑
        "literalism": 3,    # 1-5: 灵活到字面
        "empathy": 5,       # 1-5: 疏离到共情
    },
)
```

- `mission`（使命）：描述该库的用途与角色，供 reflect 在推理时参照。
- `disposition`（性情）：三个 1–5 分的性状值，刻画 reflect 生成答案时的推理倾向，会实际影响回答的口吻与审慎程度。

两者都可运行时调整，CLI 对应命令（已核实）：

```
hindsight bank disposition assistant-prod                          # 查看
hindsight bank set-disposition assistant-prod --skepticism 3 --literalism 4 --empathy 5
hindsight bank name assistant-prod "My Assistant"                  # 改名
hindsight bank mission assistant-prod "I am a helpful AI assistant" # 改使命
hindsight bank stats assistant-prod                                # 统计
hindsight bank list                                                # 全部库
```

5.3 bank 级配置项

除 mission 与 disposition 外，bank 还有若干会影响行为的配置（示例：`recall_prefer_observations`、`default_observation_scopes`、`mental_model_min_refresh_interval_seconds`），通过 bank config 端点维护（REST 路径形如 `PATCH /v1/{tenant}/banks/{bank_id}/config`），在各功能章节随用随讲。配置层级总览（全局环境变量 → 租户 → bank）见第 10 章。

删除库的操作经由客户端的完整 API 命名空间完成（Python 中为 `await client.banks.<方法>` 风格的异步调用，覆盖 SDK 辅助方法未包含的操作，方法名见官方 API Reference）。删除是破坏性操作，连同库内全部记忆与派生数据一并移除，操作前应确认备份与影响范围。

5.4 bank 模板与指令

对需要批量、标准化建档的团队，Hindsight 支持**声明式 bank 模板（bank templates）**：把库配置（使命、disposition、预置心智模型等）写成模板，按模板一键建库。官方维护模板仓库与模板中心（Bank Templates Hub），SDK 客户端亦提供 `list_mental_models`、`list_directives` 等 bank 级辅助方法管理库内派生知识与指令（directive）。模板的具体字段与示例见官方文档 bank-templates 页，本教程不展开转述，以免与后续版本细节出入。

第 6 章 检索机制详解：语义·关键词·图·时间四路检索与 budget 控制

recall 是日常使用中调用量最大、也最影响智能体回答质量的接口。本章说明它内部如何并行检索与融合排序，逐项讲清请求参数，并给出调参建议。完成本章学习后，读者能够针对召回不足、返回过载等具体症状调整对应参数。

6.1 四路检索与融合重排的整体流程

对同一个查询，Hindsight 并行运行四种检索策略，随后融合排序：

策略	检索对象	擅长
语义（semantic）	稠密向量相似度	换不同措辞也能命中的相近语义
关键词（keyword）	BM25 全文匹配	精确标识符、专有名词（如工单号 amber-17）
图（graph）	实体、时序、因果链路	关系式问题（A 和 B 怎么关联）
时间（temporal）	时间范围与时间表达式	「上个月」「六月发生了什么」

四路结果经倒数排名融合（RRF，Reciprocal Rank Fusion）合并后，再由交叉编码器（cross-encoder）重排模型按原始查询重新打分，最后按相关性顺序装填进 token 预算。整个过程对调用方透明，只需按需调整下述参数。

6.2 recall 请求参数全解

以下参数均已在官方 API 文档核实，SDK 与 REST 一致可用：

```python
results = client.recall(
    bank_id="my-bank",
    query="How are Alice and Bob connected?",
    types=["world", "observation"],   # 缩小类型范围，降低查询成本
    budget="high",                    # low / mid(默认) / high
    max_tokens=4096,                  # 返回事实的 token 预算（默认 4096）
)
```

参数	作用与要点
`query`	唯一必填项；超过 500 token 的查询被拒绝
`types`	`world` / `experience` / `observation` 的子集；每种类型独立跑完整四路检索，收窄即省成本
`budget`	控制检索深度与广度：`low` 适合快速查询，`mid` 日常默认，`high` 适合需要间接关联或穷尽覆盖的问题
`max_tokens`	结果事实合计 token 上限；超预算的过长事实被跳过而非截断；设为 `0` 表示只取扩展数据（如 chunks）不要事实
`prefer_observations`	观察优先：命中观察的原始事实被丢弃以避免重复，空出的名额由次优结果补上；默认关闭，可在请求或 bank 配置中开启
`query_timestamp`	以调用方视角锚定相对时间与新旧加成；重放历史对话、回溯查询时使用
`temporal_window`	显式给出 `{start, end}` 时间窗（ISO 8601，边界含）；注意它作用是排序而非过滤，窗外结果仍会出现
`include`	扩展数据开关：`chunks`（来源原文切片）、`source_facts`（观察的证据事实）、`entities`（默认开启）
`trace`	设为 `true` 时返回完整调试信息（各阶段耗时、四路原始命中、融合细节）
`min_scores`	各阶段得分下限；仅 `reranker`、`final` 两级可用作结果可信阈值，语义与关键词两级只约束各自检索臂（见 6.4 节）

关于 max_tokens 的设计语义：智能体按上下文窗口思考预算，而不是数结果条数，因此该参数以 token 计而非常规的 limit。全部返回结果按 `scores.final` 降序排列、按序装填直至预算用满：放不进剩余预算的长事实被跳过，排在其后的短事实仍会装入；任何有过命中的查询不会返回空——若最高分事实放不下预算，它将整体返回且允许超出，而不是被截断改写。

6.3 标签过滤与多用户隔离

单个 bank 服务多用户、多会话、多房间时，用标签控制可见性。匹配由 `tags`（标签列表）与 `tags_match`（匹配模式）共同决定：

`tags_match`	未打标签的记忆	匹配条件
`any`（默认）	包含	命中任一给定标签
`any_strict`	排除	命中任一给定标签
`all`	包含	命中全部给定标签
`all_strict`	排除	命中全部给定标签
`exact`	—	标签集合与给定集合完全相等（顺序无关，允许集合为空表示仅取全局记忆）

两个使用要点：

1. 无标签数据应放给谁看：默认模式（`any` / `all`）把未打标签的记忆视为全员可见的全局知识；若业务要求严格分区，使用 `any_strict` / `all_strict`。
2. 复合布尔逻辑用 `tag_groups`（与平铺 `tags` 互斥，不能同用）：顶层组间 AND，组内支持 `{tags, match}` 叶子与 `{and: [...]}`、`{or: [...]}`、`{not: ...}` 组合节点，叶子可设 `resolve: "fuzzy"` 用三元组相似度容错拼写偏差（阈值 0.45）。

```python
# 只看 alice 的记忆，且允许全局未打标签记忆出现
response = client.recall(
    bank_id="my-bank",
    query="communication preferences",
    tags=["user:alice"],
    tags_match="any",
)
```

6.4 结果结构与 trace 调试

每条结果（`RecallResult`）的字段沿用第 3.3 节引出的清单，重点是 `scores` 对象：

得分字段	含义
`final`	最终排序依据（重排相关性 × 近因 / 时间 / 证据加成等的合成）
`reranker`	交叉编码器归一化分数（0–1）；启用直通式重排或 TypeSafe 重排时为空
`semantic`	原始向量余弦相似度；非语义臂命中时为空
`keyword`	BM25 原始分数；非关键词臂命中时为空

两条实践建议（属建议类内容，附理由）：

1. **优先按序消费而非卡分数。**官方明确说明各分数是单次查询内的相对量，跨查询不可直接比较；先无阈值地 recall 一次观察实际分数分布，再校准任何固定阈值。
2. **观察证据链需要显式打开。**启用 `include_source_facts=True`（另配 `max_source_facts_tokens`）后，observation 类型结果会带 `source_fact_ids` 与 `source_facts` 字段对应表，可回溯每条观察的证据；预算不足时响应以 `source_facts_truncated` 提示。

排查「为什么没召回」时打开 trace（SDK `trace=True` 或 CLI `--trace`），可看到四路各自的命中、RRF 融合候选、重排后的序列与各阶段耗时，用来判断问题出在检索层还是排序层。

第 7 章 心智模型与知识页：让智能体自带「结论」

reflect 每次都要现场检索加推理，成本高且答案易变。本章介绍两类把「推理结果前置化」的派生知识：心智模型与知识页。完成本章学习后，读者能够为高频问题建立常备答案，并配置其自动更新策略。

7.1 心智模型与检索阶梯

心智模型（mental model）就是对某个问题的**常备答案**：定义一次问题，Hindsight 立即在后台运行一次 reflect 生成答案并存储，之后随着库内知识演进自动改写。读取一个心智模型只是一次数据库读：无检索、无综合、无 LLM 调用，智能体启动时加载一页既有结论即可开工。

reflect 的检索遵循三级阶梯（官方文档表格转述）：

层级	产生方式	粒度
心智模型	用户显式定义	每问题一份文档
观察	整合自动生成	每事实簇一条信念
原始事实	retain 自动抽取	每陈述一条事实

心智模型新鲜且覆盖问题时，reflect 直接取用，省去下探。心智模型还记录证据来源（based_on 引用）与历史版本，可追溯「它当时依据什么写成」。

7.2 创建与管理心智模型

```
# 创建：后台执行一次 reflect 并保存结果，返回 operation_id
result = client.create_mental_model(
    bank_id="assistant-prod",
    name="Team Communication Preferences",
    source_query="How does the team prefer to communicate?",
    tags=["team", "communication"],      # 可选：限定读取与可见范围
    # id="communication-policy",         # 可选：自定义 ID（小写字母数字与连字符）
    # trigger={...},                     # 可选：自动刷新策略（7.3 节）
)
print(result.operation_id)
```

创建参数表：

参数	必填	说明
`name`	是	可读名称
`source_query`	是	生成内容所用的问题
`id`	否	自定义 ID；已存在同 ID 时请求被拒绝
`tags`	否	同时约束刷新可读的记忆范围与 reflect 时的可见性；默认按 `all_strict`（须包含全部标签）匹配
`max_tokens`	否	内容长度上限
`trigger`	否	刷新触发器设置（refresh_after_consolidation / refresh_cron 等）

其余管理方法（均已核实）：`list_mental_models(bank_id, detail=...)`、`get_mental_model`、`update_mental_model`、`refresh_mental_model`、`clear_mental_model`（清空内容使下次刷新重建）、`delete_mental_model`、`dry_run_refresh_mental_model`（试运行，预览刷新将做出何种改动；成本与真实刷新相同）。

两个易错点（来自官方文档警告）：

1. **给模型加标签会收窄刷新可读范围**：若尚无带那些标签的记忆，刷新会返回空内容；应先补齐记忆标签，或用 `trigger.tags_match` 覆盖默认的 `all_strict`。
2. **列取默认只回元数据**：`list_mental_models` 的 `detail` 参数默认 `metadata`，需要内容时显式传 `detail="content"`。

7.3 自动刷新：触发器、模式与细节

`trigger` 是心智模型保持「永远最新」的关键配置：

设置	默认	说明
`mode`	`"full"`	`full` 每次整体重写；`delta` 只对结构化文档做增删改操作，未触及部分逐字节保留（适合长期演进的 playbook 类文档）
`refresh_after_consolidation`	false	观察整合完成后自动刷新
`refresh_cron`	null	UTC 五字段 cron 定时刷新（如 `"0 3 * * *"` 每日 03:00）；与 refresh_after_consolidation 互斥
`full_refresh_cron`	null	delta 模型的定时整体重写（如每周日一次），对冲长期小步编辑的文本漂移
`min_refresh_interval_seconds`	null	自动刷新最小间隔；间隔内的触发不丢弃而是折叠排队
`response_schema`	null	JSON Schema 结构化输出，刷新时同时产出机器可读视图
`keep_trace`	false	记录每次刷新的执行轨迹（含定时刷新），便于排查

自动刷新内置安全和节流机制，实例：

```
client.update_mental_model(
    bank_id="assistant-prod",
    mental_model_id=model_id,
    trigger={
        "mode": "delta",
        "refresh_after_consolidation": True,
        "min_refresh_interval_seconds": 1800,   # 半小时最多刷新一次，触发折叠排队
    },
)
```

刷新前会先做时效性检查：仅当该模型自身作用域内确有新记忆落库才执行，且「什么都没读到」时不出 LLM 调用直接完成；失败刷新由 worker 重试（`HINDSIGHT_API_WORKER_MAX_RETRIES`，默认 3 次）后暂停自动触发，直至某次手动刷新成功后恢复。刷新全程受 `HINDSIGHT_API_REFLECT_WALL_TIMEOUT` 约束。判定新旧用的是两个时间戳：`last_refreshed_at`（上次刷新何时完成）与 `last_memory_seen_at`（上次刷新实际读到的最新记忆），列表与详情接口返回的 `is_stale` 即由二者比较得出。

7.4 知识页（knowledge pages）与整库知识树

知识页是心智模型的「/wiki 化」封装：一个 bank 自我书写的活文档集合，按文件夹组织、可搜索、可整体导出，还能镜像到磁盘当普通 Markdown 用。用法与心智模型一致——给名字给问题，其余取默认。CLI 提供完整管理命令：

```
hindsight knowledge-base tree my-bank                          # 查看文件夹与页面树（含过期标记）
hindsight knowledge-base create-folder my-bank "Runbooks"      # 建目录
hindsight knowledge-base create-page my-bank \
  "Deploying the API" "How is the API deployed?" \
  --parent-id "$FOLDER_ID" --tags ops,type:runbook             # 建页，内容后台生成
hindsight knowledge-base get-page my-bank "$PAGE_ID"           # 读取页面（Markdown）
hindsight knowledge-base search my-bank "how do we deploy" --limit 5   # 全文 + 向量混合检索
hindsight knowledge-base update my-bank "$PAGE_ID" --source-query "New question?"  # 改章定义
hindsight knowledge-base export my-bank                        # 整树导出 Markdown
hindsight knowledge-base delete my-bank "$NODE_ID" -y          # 删除任意节点及其子树
```

还有一个在禁止常驻命令行场景很实用的能力：`hindsight fs mount --bank <bank_id>` 把知识库镜像为磁盘上的真实 Markdown 文件，由后台刷新环保持最新，可直接用 grep、编辑器访问。1.1.0 起知识页与心智模型还可下载为 PDF 或 Markdown 文件。

心智模型与知识页是同一事物的两种形态：每个知识页背后就是一个心智模型（列取接口以 `is_knowledge_page` 区分）。需要嵌入智能体启动流程时用心智模型接口；需要人为浏览维护时用知识页与 CLI。

第 8 章 集成实战：LiteLLM Wrapper、LangGraph 等框架、MCP、编码智能体

把记忆接入智能体有两条路径：显式调用 retains / recall / reflect 自行编排，或者使用封装好的集成让记忆自动发生。本章按「最省代码到最深入」的顺序覆盖各接入方式。完成本章学习后，读者能够为自己的技术栈选择集成方式并完成接入。

8.1 选择接入路径

路径	代码量	控制粒度	适用
LLM Wrapper（LiteLLM）	约 2 行	会话级自动	已有 LLM 调用，想零改动获得记忆
框架集成	依框架而定	框架生命周期内自动	LangGraph、CrewAI 等既有智能体
MCP Server	零代码	工具调用	任意支持 MCP 的客户端
编码智能体集成	一条安装命令	会话自动	CLI 编码助手长期项目记忆
显式 SDK / REST API	最多	每次调用完全可控	需精细编排流水线的应用

8.2 LLM Wrapper：两行代码接入记忆

`hindsight-litellm` 包提供面向 OpenAI 与 Anthropic SDK 的包装器：

```
pip install hindsight-litellm
```

```
from openai import OpenAI
from hindsight_litellm import wrap_openai

client = wrap_openai(
    OpenAI(),
    bank_id="user-123",
    hindsight_api_url="http://localhost:8888",  # 自建服务地址；不带此参数则默认 Hindsight Cloud
)

# 调用方式与原 SDK 完全一致
response = client.chat.completions.create(
    model="gpt-5-mini",
    messages=[{"role": "user", "content": "What do you know about me?"}],
)
```

工作方式：每次调用前，包装器先向对应 bank recall 相关记忆并注入请求；调用结束后自动 retain 本轮对话。Anthropic SDK 使用 `wrap_anthropic()`。所有配置（bank、recall 预算、事实类型、是否改用 reflect 取代 recall）都可通过调用时的 `hindsight_*` 关键字参数覆盖。底层是 LiteLLM，因此同一套包装器覆盖 100 余种模型，包括经由网关与 Claude、Gemini 等服务。若需要显式控制「何时存、何时取」，改用 SDK 直接调用（第 3 章方式）。

8.3 框架与平台集成一览

官方维护集成中心（hindsight.vectorize.io/integrations），已核实的主要集成如下；多数集成为不改代码或几乎不改代码的接入：

类别	集成对象
编码智能体	Claude Code、Codex、Cursor、GitHub Copilot、opencode、Cline、Aider、Zed、Continue、Roo Code、OpenHands
智能体框架	LangGraph / LangChain、LlamaIndex、CrewAI、Pydantic AI、OpenAI Agents SDK、Google ADK、Agno、Strands、AutoGen、Microsoft Agent Framework、Vercel AI SDK、Haystack
低代码 / 无代码	n8n、Zapier、Dify、Flowise
应用与工具	ChatGPT、Perplexity、Obsidian、Pipecat、Vapi

各集成的安装与配置方法差异较大，均以集成中心对应页面为准；本教程不逐一转录代码，避免与各集成自身的版本节奏脱节。

8.4 MCP Server：把记忆暴露为工具

每个 Hindsight 服务内置 Model Context Protocol（MCP，模型上下文协议）端点，**每个 bank 一个，默认启用**：

```
http://localhost:8888/mcp/{bank_id}/
```

把任意 MCP 客户端指向该地址，即可把 retain、recall、reflect 作为工具暴露给模型。这一路径的典型场景：桌面 AI 助手、支持 MCP 的 IDE 插件等拿到「记住这条 / 找出相关记忆 / 综合回答已有知识」三个工具后，即可自行决定何时使用。MCP 侧产生的访问可在审计日志中按传输方式过滤（`hindsight audit list my-bank --action recall --transport mcp`）。

（MCP 为开放标准，其定义与工具调用语义超出本教程范围；此处只关心 Hindsight 暴露的端点与工具集。）

8.5 编码智能体的项目记忆

面向 Claude Code、Codex CLI、Cursor CLI、GitHub Copilot CLI、opencode 等命令行编码智能体，官方提供一体化安装包，为每个代码仓库自动构建专用 bank：

```
npx @vectorize-io/hindsight-coding-agents install all          # 检测并接入全部已装智能体
npx @vectorize-io/hindsight-coding-agents install claude-code  # 只接入单个
```

工作机制：从 git 历史与会话记录自动摄取（无需手动初始化命令），智能体开工时把本仓库记忆注入上下文，另以 curated 知识页沉淀架构、约定与进行中事项。支持的智能体清单（已核实的部分）：Claude Code、Codex CLI、Cursor CLI、GitHub Copilot CLI、opencode、Kilo CLI、Cline CLI、Antigravity CLI、Devin CLI、pi、Prime Agent、Grok Build、DeepSeek Harness（以集成页为准）。

另外，编码场景下直接开发 Hindsight 本身相关代码时，可为编辑器安装官方文档技能包：

```
npx skills add https://github.com/vectorize-io/hindsight --skill hindsight-docs
```

8.6 显式接入：REST API 与生成的客户端

需要完全控制调用序列（例如按业务事件精确决定写入时机）时，直接面向 REST API 开发：全部端点与请求模式见官方 API Reference（/api-reference）。Python 客户端同时把完整生成的 API 暴露为异步命名空间（`client.banks`、`client.memory`、`client.documents`、`client.mental_models`、`client.directives` 等），其方法需要 await 辅助方法未覆盖的所有操作。

接入方式选择的建议：从「会自动发生」到「按需发生」逐档评估——如果记忆对助手是透明的背景能力，用 Wrapper 或 MCP；如果记忆本身就是业务逻辑的一部分（客服工单关联、项目管理复盘），用显式 SDK 编排并在关键节点写入。

第三篇 精通篇：用精

第 9 章 生产级部署：Compose、Helm、Cloud、Oracle 与存储选型

开发环境跑通之后，生产部署要回答四个问题：数据库放在哪里、容量如何伸缩、要不要自管运维、数据合规是否满足。本章按自管程度从低到高覆盖各选项，并给出存储选型的判断依据。完成本章学习后，读者能够选定一条部署路线并规划升级路径。

9.1 部署演进路径

阶段	推荐形态	理由
原型验证	Docker 全量镜像（内嵌数据库）	一条命令，无外部依赖
小规模生产	Compose + 外部托管 PostgreSQL（Supabase、Neon 等）	数据库托管化，服务仍简单
中大规模生产	Kubernetes + Helm（含独立 worker 扩缩）	弹性伸缩与高可用
不自管运维	Hindsight Cloud 或 Enterprise	托管加面板加 SLA

9.2 Docker Compose 与外部 PostgreSQL

仓库自带 Compose 示例（docker 目录下的 docker-compose 子目录）。使用外部数据库的最小流程：

```
export OPENAI_API_KEY=sk-xxx
export HINDSIGHT_DB_PASSWORD=choose-a-password
cd docker/docker-compose
docker compose up
```

要点：LLM 密钥与数据库口令经环境变量传入（服务内部配置项为 `HINDSIGHT_API_DATABASE_URL`，Compose 模板负责桥接）。数据库须为 PostgreSQL 14 及以上并启用向量扩展（9.5 节）。

9.3 Kubernetes 与 Helm chart

官方 chart 以 OCI 制品发布，命令（官方安装文档原文）：

```
helm install hindsight oci://ghcr.io/vectorize-io/charts/hindsight \
  --set api.llm.provider=groq \
  --set api.llm.apiKey=gsk_xxxxxxxxxxx \
  --set postgresql.enabled=true
```

外接数据库时改为 `--set postgresql.enabled=false --set api.database.url=postgresql://user:pass@postgres.example.com:5432/hindsight`；固定版本用 `--version 0.1.3` 形式；升级用 `helm upgrade`。要求 Kubernetes 集群（GKE、EKS、AKS、自建均可）与 Helm 3.8 及以上。

独立 worker 扩缩：高吞吐场景把后台任务（异步 retain 处理、整合、刷新）拆出独立副本：

```
helm install hindsight oci://ghcr.io/vectorize-io/charts/hindsight \
  --set worker.enabled=true \
  --set worker.replicaCount=3
```

chart 以 StatefulSet 部署 worker，Pod 名稳定（如 `hindsight-worker-0`）且天然充当稳定 worker 身份；若自建普通 Deployment，必须为每个副本显式设置不同的 `HINDSIGHT_API_WORKER_ID`，否则任务归属会随主机名漂移而失联。全部 chart 参数见仓库 helm/hindsight/values.yaml。

9.4 Hindsight Cloud 与企业版

托管版本面向不想管理基础设施的团队：按用量计费（含初始免费额度）、提供管理面板、备份、团队协作与 99.9% 可用性 SLA。接入只需把客户端指向托管地址：

```
from hindsight_client import Hindsight

client = Hindsight(base_url="https://api.hindsight.vectorize.io", api_key="hsk_...")
```

自部署与托管的选型判断：数据必须完全驻留自有基础设施、或需要深度定制扩展点（第 12 章、官方 extensions 文档）时选自部署；追求运维成本最小化、可接受容量由托管方弹性处理时选 Cloud。另据官方文档，2026 年 10 月发布了 Memlake（面向对象存储、宣称亿级记忆 p99 毫秒级检索）方向的新架构，相关能力以官方博客与发布说明为准。

9.5 存储选型与向量扩展

所有部署形态（Cloud 除外）后端都是关系数据库加向量扩展，官方支持的组合：

数据库	向量扩展	说明
PostgreSQL 14+	pgvector（默认）	常规选择
PostgreSQL 14+	pgvectorscale	官方生态的高性能扩展之一
PostgreSQL 14+	vchord	同上
Google AlloyDB	scann	Google 云配套
Oracle AI Database 23ai	内置向量能力	与 PostgreSQL 版本功能对等，面向企业既有 Oracle 环境

以 `HINDSIGHT_API_VECTOR_EXTENSION` 选择扩展。托管数据库中，Supabase、Neon、Azure Database for PostgreSQL、AWS RDS / Cloud SQL 均在官方验证之列（自建时扩展须由管理员预装，`CREATE EXTENSION vector;`）。选型建议：优先选操作团队最熟悉的关系数据库与托管服务；向量扩展的差异主要在索引性能与内存占用，需要对齐体量后压测验证，不宜默认。

第 10 章 配置体系、监控与日常运维

服务上线之后，日常问题从「能不能跑」转变为「跑得稳不稳」。本章覆盖配置层级、观测手段与三类最高频的运维操作。完成本章学习后，读者能够定位配置生效层级、读取关键指标，并处理卡住的后台任务。

10.1 配置层级与常用环境变量

配置按三级覆盖，越具体优先级越高：

```
全局环境变量（HINDSIGHT_API_*）→ 租户级配置 → bank 级配置（bank config）
```

日常使用最频的环境变量（均已核实）：

变量	作用
`HINDSIGHT_API_LLM_PROVIDER` / `HINDSIGHT_API_LLM_API_KEY` / `HINDSIGHT_API_LLM_MODEL` / `HINDSIGHT_API_LLM_BASE_URL`	LLM 提供商四件套
`HINDSIGHT_API_DATABASE_URL`	外部数据库连接串
`HINDSIGHT_API_VECTOR_EXTENSION`	向量扩展选择
`HINDSIGHT_API_EMBEDDINGS_PROVIDER` / `HINDSIGHT_API_EMBEDDINGS_LOCAL_MODEL`	嵌入服务（本地模型名示例 BAAI/bge-small-en-v1.5）
`HINDSIGHT_API_RERANKER_PROVIDER`	重排服务（本地可用 flashrank）
`HINDSIGHT_API_WORKER_ID`	worker 身份标识（生产必须设稳定值）
`HINDSIGHT_API_WORKER_MAX_RETRIES`	后台任务失败重试次数（默认 3）
`HINDSIGHT_API_RETAIN_BATCH_ENABLED` / `HINDSIGHT_API_RETAIN_BATCH_TOKENS`	异步写入批量接口开关 / 单文档切片 token 上限
`HINDSIGHT_API_FILE_STORAGE_TYPE`	上传文件存储位置（默认 PostgreSQL，可 S3 / GCS / Azure）
`HINDSIGHT_API_RECALL_PREFER_OBSERVATIONS`	recall 观察优先的部署默认值
`HINDSIGHT_API_MENTAL_MODEL_MIN_REFRESH_INTERVAL_SECONDS`	心智模型最小刷新间隔（全局默认）
`HINDSIGHT_API_REFLECT_WALL_TIMEOUT`	单次 reflect / 刷新整体时间上限

bank 级覆盖项见第 5.3 节；完整清单见官方 Configuration 文档（此处仅为最高频子集）。

CLI 自身配置：`hindsight configure`、命名 profile（`hindsight profile create prod --api-url ... --api-key ...`；`-p prod` 或 `HINDSIGHT_PROFILE` 选用；Unix 下配置文件以 0600 权限保存）。优先级：环境变量 > 显式 `-p` > 共享配置 `~/.hindsight/config` > 默认 localhost:8888。

10.2 监控：指标与请求追踪

生产观测手段有两层：

1. **Prometheus 指标**：内置面向 LLM 调用、token 用量与延迟的指标；官方仓库 monitoring/grafana/dashboards 目录提供现成 Grafana 面板。HTTP 指标按稳定路由模板标注，聚合口径稳定。
2. **请求历史与追踪**：1.1.0 起请求历史还包括嵌入、重排与决策模型的调用记录，可在控制台查看；recall 的 `trace` 参数与心智模型的 `keep_trace` 则用于单次请求级别的深查（第 6.4、7.3 节）。

日常巡检建议关注：后台任务积压（operation list）、LLM token 成本走势、p99 延迟、整合与刷新失败次数。

10.3 日常运维命令

操作与队列管理（所有异步任务都会在 operation 体系留档）：

```
hindsight operation list my-bank                 # 列出后台操作
hindsight operation get my-bank "$OPERATION_ID"  # 查看状态（等待原因、重试时间等）
hindsight operation cancel my-bank "$OPERATION_ID"
hindsight operation retry my-bank "$OPERATION_ID"
```

卡住任务恢复：整合卡住用 `hindsight bank consolidation-recover my-bank`；更大范围的僵尸操作诊断与按 worker 身份回收的做法见官方 admin CLI 文档，前提是部署时已按第 9 章建议固定了 `HINDSIGHT_API_WORKER_ID`。

审计日志：

```
hindsight audit list my-bank --action recall --transport mcp     # 按动作与入口过滤
hindsight audit list my-bank --start-date "2026-04-01T00:00:00Z"  # 时间范围
hindsight audit get my-bank "$LOG_ID"                             # 查看完整请求 / 响应
```

Webhooks（把生命周期事件推送出来，接自建告警或工作流）：

```
hindsight webhook list my-bank
hindsight webhook create my-bank https://example.com/hook \
  --event-types retain.completed,consolidation.completed \
  --secret my-hmac-secret
hindsight webhook deliveries my-bank "$WEBHOOK_ID"   # 投递历史
hindsight webhook delete my-bank "$WEBHOOK_ID" -y
```

已核实的事件类型包括 retain 相关（如 `retain.completed`）、整合相关（如 `consolidation.completed`）、刷新生命周期事件，以及安全类 `memory_defense.triggered`（第 11 章）；全量事件清单见官方 webhooks 文档。

备份与迁移：官方支持把 bank 导出为归档并再次导入（大表分页处理，导入按批次进行的机制在变更日志中有记录）；版本间的破坏性变更与升级注意事项见附录 FAQ。备份方案应以归档导出加数据库层常规备份（含向量数据）组合为宜——哪个更完整取决于部署自管程度，此处为建议性判断。

10.4 升级流程建议

由于文档与发布均按版本线管理（官方文档提供 1.1、1.0、0.10、0.9 多版本），升级前后应完成：

1. 阅读目标版本的变更日志中 breaking changes 与数据库迁移项（例如 1.1.0 对 `memory_units` 高容量表的迁移要求升级窗口更宽松）。
2. 数据库迁移在服务启动时自动执行，先在低峰期用副本演练。
3. 同步升级客户端包与控制台镜像，避免旧客户端读不到新字段。

第 11 章 安全合规：Memory Defense 与 PII 防护

记忆库天然聚集长期数据：用户诉求、对话原文引用、项目信息。对任何要长期保存交互数据的系统，敏感信息进入存储是必须主动设防的问题。本章覆盖 Hindsight 的内置防线与访问控制要点。完成本章学习后，读者能够为 bank 启用并验证敏感信息脱敏策略。

11.1 Memory Defense：写入前的脱敏扫描

Memory Defense 是按 bank 启用的可选机制：开启后，每条写入该 bank 的内容在落库前 matched 45 组正则模式（涵盖主流云与厂商密钥、支付、通讯、连接串、私钥、JWT、美国侧 PII 等），命中部分替换为 `[REDACTED:<类型>]` 标记；被清洗后的版本才是真正入库的内容，后续 recall、导出、reflect 看到的都是清洗版。

两种动作：

- `redact`（默认推荐）：把匹配片段替换为 `[REDACTED:github_token]` 这类标记后入库。
- `block`：整条含命中的条目直接拒收；若一次 retain 的全部条目都被拒，返回 HTTP 422。

配置方式（bank 级，写入 config）：

```
{
  "memory_defense": {
    "enabled": true,
    "rules": [
      { "on": "sensitive_data", "action": "redact" }
    ]
  }
}
```

模式覆盖示例（完整 45 项清单见官方 Memory Defense 文档）：

标签	匹配对象
`aws_access_key`	`AKIA` 开头的 Amazon 访问密钥
`github_token`	`ghp_` 等 GitHub 令牌
`openai_key`	`sk-` 系 OpenAI 密钥
`groq_key`	`gsk_` 系列
`stripe_secret`	`sk_live_`、`sk_test_`
`slack_token`	`xoxb-` 等
`db_url_postgres`	`postgres://user:pass@host` 连接串
`private_key_pem`	PEM 私钥块
`jwt`	JWT 三段式令牌
`credit_card`、`ssn_us`	卡号与美国社会安全号

三个必须清楚的边界：

1. **只对开启之后的写入生效。**既有记忆不会被回溯扫描；若需要清洗存量数据，须重新摄取或手工清理（官方文档明示）。
2. 触发脱敏或拦截时，订阅了 `memory_defense.triggered` 的 webhook 会收到事件（含动作、文档 ID、命中模式），可转发 SIEM 或即时通讯告警；开启审计后并在审计日志中以 `memory_defense` 条目留痕。
3. 该功能对仓库入口之外的数据流（例如直接写数据库的场景）不构成保护，仍属纵深防御的一层。

11.2 访问控制与镜像校验

已核实的访问控制手段：

1. **API Key**：服务端开启鉴权后，客户端以 Bearer Token 携带密钥（`Hindsight(base_url=..., api_key=...)`），CLI 用 `HINDSIGHT_API_KEY` / `hindsight configure --api-key` 配置；tag 范围调用者会被限制在对应标签范围的数据可见性内（1.0.0 起对全 bank 数据的操作被拒绝）。
2. **控制台访问密钥**：环境变量 `HINDSIGHT_CP_ACCESS_KEY` 设置后，登录控制台须输入。
3. **标签可见性**：第 6.3 节的 tags 机制是数据面最小化的主要工具；对多租户数据隔离要求高的场景应叠加 5.1 节的库隔离。
4. **镜像签名校验**：所有发布镜像带 Cosign 签名，可按官方安装文档校验命令行验证来源。

数据删除相关风险提示：`document delete`、`bank delete`、`clear-observations` 均为不可恢复操作；API 也没有软删除机制承载误删恢复（以官方文档为准）。生产操作前建议先完成第 10.3 节的备份验证流程。

第 12 章 性能与成本调优

Hindsight 性能画像有一组基本事实：写入成本在后台（异步抽取、嵌入）、检索延迟由四路并行与重排决定、LLM 调用发生在 retain 与 reflect 两处。调优就是把这三段资源配置到与应用匹配的水平。本章给出官方核实过的影响因素与可操作措施。完成本章学习后，读者能够定位自己部署当前的瓶颈并按清单优化。

12.1 性能相关的体积与形态取舍

从第 2 章的硬件表得出的决策面：

选项	影响
全量镜像 vs slim	全量内置本地嵌入与重排（内存约 0.8–1 GB 稳态，高峰 1.2–1.5 GB）；slim 约 512 MB 量级，但依赖外部服务
CPU vs GPU	本地重排（交叉编码器）是查询延迟的主要瓶颈，生产建议 GPU 或外部重排服务
独立 worker	后台抽取密集时段不再抢占在线 API 的资源
NVIDIA GPU 版镜像	官方提供 CUDA 构建配方（自建镜像，盘面代价约 11 GB），仅在确实具备 GPU 时值得

12.2 检索与 reflect 的调参

症状	建议
检索慢	`budget` 降为 `mid` 或 `low`；`types` 收窄；`max_tokens` 收紧；控制台观察 trace 各阶段耗时
返回过多挤占上下文	`max_tokens` 上限对齐智能体预留的上下文份额；或开 `prefer_observations` 用一条观察顶替多条事实
找不准	`budget="high"`；必要时 `types` 放开复查；检查写入时 context 与 timestamp 质量
reflect 太慢	reflect 常态优先心智模型；把高频问题固化为心智模型（`refresh_after_consolidation`），把 reflect 留给低频复杂问题

查询构造的实践要点（依据官方文档的检索语义整理）：最多过滤携带语义的前提，查询本身保持自然语言；时间类问题靠 `temporal_window` 或 `query_timestamp` 锚定；意图明确的标识符（工单号、文档名）直接作为查询即可命中关键词路。

12.3 写入侧的成本优化

按影响从小到大排列（合计或可达数十个百分点，具体幅度取决于负载构成）：

1. **异步 retain 批量化**：合并单发为批量，降低网络与调度开销；
2. **增量式追加**：持续增长的内容用 `update_mode="append"`，已被处理过的切片不会重复抽取；
3. **提供商批量接口**：`HINDSIGHT_API_RETAIN_BATCH_ENABLED=true` 后抽取得以 50% 左右折扣价处理（仅 OpenAI / Groq / Gemini，且须异步模式）；
4. **心智模型节流**：`min_refresh_interval_seconds` 把高频触发的刷新折叠为一次；`mode:"delta"` 只改未覆盖部分；二者对 LLM 成本影响显著；
5. **决策模型（decision model，1.1.0 实验特性）**：为整合、排序与刷新等环节配置轻量决策模型先行分流，可减少大模型调用；属实验能力，启用前先在小环境验证。

12.4 效果核验与官方评测资源

上述措施的验证与评测在官方体系中有两个入口：一是 benchmarks.hindsight.vectorize.io 的公开评测（含每模型精度、延迟与成本数据），二是官方 best practices / cookbook 文档。为特定负载做容量规划时，建议以自身流量回放压测为准（此处为建议，官方亦提供 system evals 工程目录可参考其做法）。

第 13 章 综合案例：三类智能体设计

前三篇的材料至此覆盖了从概念到运维的完整链条。本章把它们组装回三个典型业务场景，形成从库设计到上线自检的端到端方案。三个案例共用一套骨架，差异在库边界与记忆策略；这就决定了案例主体是「设计推理」，代码只做示意。

13.1 共同系统骨架

三个案例的系统骨架相同：应用后端持有 Python 客户端，按调用bject驱动 retain；供模型上下文用的是 recall 结果注入系统提示词，reflect 在重问题路径上使用；派生知识（心智模型 / 知识页）在启动时预载；webhook 把敏感信息触发事件转给安全通道。

带记忆对话的最小实现模式：

```python
from openai import OpenAI
from hindsight_client import Hindsight

llm = OpenAI()
mem = Hindsight(base_url="http://localhost:8888")

def chat(bank_id: str, session_tag: str, user_text: str) -> str:
    # 1. 取回相关记忆（用户标签限定可见范围）
    rec = mem.recall(
        bank_id=bank_id,
        query=user_text,
        types=["world", "experience", "observation"],
        tags=[session_tag] if session_tag else None,
    )
    memories = "\n".join(f"- {r.text}" for r in rec.results)

    # 2. 组装上下文后交给 LLM
    messages = [
        {"role": "system", "content": f"Relevant memories:\n{memories}"},
        {"role": "user", "content": user_text},
    ]
    reply = llm.chat.completions.create(...)

    # 3. 会话结束后写入完整轮次（含 document_id 保证幂等）
    mem.retain(
        bank_id=bank_id,
        content=f"User: {user_text}\nAssistant: {reply.choices[0].message.content}",
        context="conversation",
        document_id=f"turn-{session_tag}",
    )
    return reply.choices[0].message.content
```

（示例中第 2 步省略了具体模型调用，接入任意模型均可；追求自动记忆可直接采用第 8.2 节的 wrap_openai 一体化方案，此处保留显式写法是为了引出三个设计点：检索类型组合、标签可见性、幂等写入。）

13.2 案例 A：个人助理智能体

目标：记住用户偏好、在途事项与约定，跨会话延续人设。库按用户划分（`user-<uid>`），写入策略：每轮对话写入（document_id 按轮递增，实现幂等重建），任务类信息（日程、承诺）单独条目并打 `topic:<主题>` 标签。**心智模型**：`用户偏好`（question：「What are this user's preferences?」，`refresh_after_consolidation: true`）、`在途事项`（question：「What is currently pending or in progress?」，`refresh_cron` 每小时）。启动流程把这两个心智模型读进系统提示词。**安全**：启用 Memory Defense redact，避免密钥、证件号进库；写入前用 metadata 记录会话来源。

13.3 案例 B：客服坐席记忆

目标：坐席多会话间保持工单历史连续，沉淀「这个客户的问题刚开始我们是怎么解的」。库按工单或公司域划分（同一 bank 由多名坐席共享，坐席动作打 `agent:<id>` 标签，tags_match 用 `any` 以保持全局知识可见）。**写入**：每轮通话记录一个 document；结案时写一条纯文本总结（`context="case closure"`）。**观察作用域**：按域（`domain:<key>`）整合观察，客户痛点随证据积累收敛。** reflect 用途**：结案质量复盘（「为什么这个客户的问题如此难处理」），以及知识库缺口识别（官方给出的支持场景之一）。KPI 口径：工单重开率（对照记忆开启前后）、坐席首响应时长（recall 延迟纳入计算，基线以官方基准页与自身压测为准）。

13.4 案例 C：项目协作智能体（AI 项目经理）

目标：项目状态、风险、约定与在途工作随时间累积，为项目经理提供随时可问的「当前态势」。库按项目划分（`project-<name>`）。**写入**：周会纪要整篇写入（带发生时间）；任务管理系统变更经 webhook / 定时任务写入异常事件（`context="jira event"`）；决策记录写入传统文档系统外的独立 retain（`context="decision"`）。**知识页树**：`architecture`（架构约定）、`conventions`（流程约定）、`risk-register`（风险登记，question：「What risks need mitigation right now?」，`refresh_after_consolidation: true` + `min_refresh_interval_seconds: 3600` 用 delta 模式增量更新）。启动流程：项目经理智能体启动时读取 `knowledge-base tree` 与关键页，形成态势首页；结项时用 `knowledge-base export` 导出整树，作为结项文档的一部分。

13.5 上线自检清单

序号	检查项	对应章节
1	部署形态与生产要求一致（数据库、worker 身份、密钥管理）	第 9、10.1
2	镜像 / 收集的版本与变更日结核对无破坏性变更影响	第 10.4、附录 FAQ
3	bank 边界与命名符合隔离设计，无保留前缀	第 5.1
4	高频问题已有心智模型或知识页，不依赖现场 reflect	第 7
5	标签可见性按最小化原则配置（多用户场景必查）	第 6.3
6	Memory Defense 已在会写入用户内容的库上启用，并做过注入测试（写入含密钥内容后用 recall 验证已脱敏）	第 11.1
7	监控接入并覆盖 token 成本、后台积压、p99 延迟	第 10.2
8	备份与恢复流程验证过一次实际演练	第 10.3
9	LLM 预算与调用路径（批量接口、节流、delta 刷新）已配置	第 12.3
10	客户端包版本与服务端版本匹配	第 10.4

完成全部检查项后，这套系统即达到可长期运行的基线状态。第九至十二章总结的运维与调优活动，按月或按事件驱动的节奏持续执行。

附录

附录 A 术语表

术语	英文	定义
记忆库	bank	隔离的记忆存储单元，即一个「大脑」，库间严格隔离
写入	retain	提交原始内容并由系统抽取事实、实体与时序索引的操作
检索	recall	并行执行语义、关键词、图、时间四路检索并融合排序的操作
综合推理	reflect	在记忆之上做多步推理并生成完整回答的操作
世界事实	world fact	关于世界的客观事实（third-person 陈述类记忆）
经历	experience	智能体自身经历过的事件与对话记录
观察	observation	由多条事实整合生成、带证据引用与证据计数的信念
心智模型	mental model	针对一个问题的常备答案文档，随知识演进自动改写
知识页	knowledge page	以文件夹组织的活文档集合，背后即心智模型
使命	mission	描述 bank 用途与角色的文字，参与推理
性情	disposition	怀疑度、字面程度、共情度三个 1–5 分的推理倾向
文档	document	一次写入内容的逻辑分组单位，是管理与删除的粒度
标签	tag	控制记忆可见范围的标记，配合 tags_match 过滤
整合	consolidation	后台把事实簇加工为观察的任务过程
作用域	scope	观察整合时的标签边界（observation_scopes）
倒数排名融合	RRF (Reciprocal Rank Fusion)	多路检索结果按排名融合的算法
交叉编码器	cross-encoder	对查询与候选逐对打分的重排模型
预算	budget	recall 参数，控制检索深度与广度（low / mid / high）
内嵌数据库	pg0	随服务自动启动的本地 PostgreSQL（默认开发配置）
后台任务进程	worker	执行异步抽取、整合、刷新等任务的服务组件
操作	operation	一次可异步执行并为追踪留档的单位（写入批次、整合、刷新等）
记忆防线	Memory Defense	按 bank 启用的敏感信息脱敏与拦截机制
决策模型	decision model	1.1.0 起引入的实验特性，用于整合、排序、刷新的轻量分流
审计日志	audit log	按动作、传输方式记录操作轨迹的日志体系

附录 B 配置与命令速查

常用环境变量（服务端，均带 `HINDSIGHT_API_` 前缀；完整清单见官方配置文档）：

```
LLM:          LLM_PROVIDER / LLM_API_KEY / LLM_MODEL / LLM_BASE_URL
数据库:       DATABASE_URL / VECTOR_EXTENSION
嵌入重排:     EMBEDDINGS_PROVIDER / EMBEDDINGS_LOCAL_MODEL / RERANKER_PROVIDER
文件:         FILE_STORAGE_TYPE
写入:         RETAIN_BATCH_ENABLED / RETAIN_BATCH_TOKENS
检索:         RECALL_PREFER_OBSERVATIONS
刷新:         MENTAL_MODEL_MIN_REFRESH_INTERVAL_SECONDS / REFLECT_WALL_TIMEOUT
运维:         WORKER_ID / WORKER_MAX_RETRIES
```

常用命令对照（场景 → Python SDK → CLI）：

场景	Python SDK	CLI
启动服务（内嵌）	`HindsightServer(...)` / `HindsightEmbedded(...)`	`hindsight-api`
写入	`client.retain(...)`	`hindsight memory retain <bank> <内容>`
批量 / 文件	`retain_batch(...)` / `retain_files(...)`	`hindsight memory retain-files <bank> <文件>`
检索	`client.recall(...)`	`hindsight memory recall <bank> <查询>`
推理	`client.reflect(...)`	`hindsight memory reflect <bank> <问题>`
建库配置	`create_bank(...)`	`hindsight bank name
库统计	（client.banks 命名空间）	`hindsight bank stats
心智模型	`create_mental_model(...)` 等	`hindsight knowledge-base ...` 配套控制台
文档管理	（client.documents 命名空间）	`hindsight document list
操作队列	（operations 端点）	`hindsight operation list
Webhooks	（webhooks 端点）	`hindsight webhook list
控制台 / 浏览器	—	`hindsight ui` / `hindsight explore`
知识库挂盘	—	`hindsight fs mount --bank <bank_id>`

附录 C FAQ

1. **Hindsight 与 RAG 是什么关系，能互相替代吗。** 两者定位不同：RAG 面向静态文档问答，Hindsight 面向随交互演进的智能体长期记忆（对比见 1.4 节）。文档问答场景沿用 RAG 更直接；记忆场景选 Hindsight。也有人把两者叠加使用（文档库用 RAG，用户会话记忆用 Hindsight），依据是数据性质不同（第 1 章中的对比表与建议）。

2. **应该为一个用户建一个库，还是多个用户共用一个库。** 首选 per-user 建库（隔离最强、观察与心智模型不会混入他人内容）；确需共享时（如团队共享项目库），用标签可见性（6.3 节）配合观察作用域（4.2 节）控制边界。

3. **中文场景需要注意什么。** 语言自动检测并保留，实体保存原文字形（「张伟」不转写为「Zhang Wei」），查询中英皆可；无需额外配置（官方多语言文档确认）。

4. **数据库有什么硬性要求。** PostgreSQL 14 及以上并启用向量扩展（pgvector 默认，另有 pgvectorscale、vchord、scann / AlloyDB，或 Oracle 23ai 对等版本）；内嵌 pg0 仅建议开发使用。

5. **全量包体积大、内存占用高，怎么办。** 改用 slim 包（`hindsight-api-slim` / `hindsight-all-slim`）并接外部嵌入与重排服务；检索延迟受本地重排影响的生产部署可改外部重排或加 GPU（12.1 节取舍分析）。

6. **内嵌数据库的数据在哪，升级换形态需要注意什么。** pg0 数据在 `~/.hindsight/data/`（Docker 镜像在容器内 `/home/hindsight/.pg0`，应用挂载卷持久化）。换形态时把连接串指向外部 PostgreSQL 即可沿用数据；卸载前先完成第 10.3 节备份。

7. **版本升级有哪些近期破坏性变更。** 已核实的三条：1.1.0 起记忆库导入拒绝非法与下划线保留前缀的 bank ID；1.0.0 起带标签范围的调用者不得执行返回全 bank 数据的操作；0.10.3 移除了 worker 的 `--max-retries` 与 `--log-level` 命令行参数（改经环境变量配置）。升级前逐版本对照官方变更日志为准。

8. **Hindsight Cloud 如何计费。** 用量计费含初始免费额度，无固定月费或按席位费（官方 pricing 页说明）；自部署、Cloud、Enterprise 的对比也在该页。

9. **遇到问题向哪里求助。** 官方文档 FAQ 与 Best Practices 页、Cookbook 案例库、GitHub Issues、Vectorize Slack 社区；此外官方 Academy 提供系统化课程（learn.hindsight.vectorize.io）。

附录 D 故障排查

症状	排查与处置
安装 `hindsight-all` 后异常或版本回退	确认是否 Intel Mac；是则改装 `hindsight-all-slim` 并接外部嵌入重排（2.7 节平台表）
Docker 启动报内嵌数据库 Permission denied	主机目录属主须为 UID 1000，或改用命名卷（2.3 节说明）
启动报 `KeyError: 'getpwuid()'` 类错误	不要用 `--user` 改运行 UID；按 UID 1000 与目录属主方案处理
recall 返回为空	用 `trace=true` 检查；核对 bank_id 拼写；确认未误用 `any_strict` 类严格匹配过滤掉未标签数据；确认后台处理已完成（operation 状态）
retain 成功但检索不到	异步处理需时间，查 `operation get`；检查是否开了 observations 优先导致事实被折叠；核对标签过滤
reflect 很慢或超时	优先心智模型方案（7 章）；调低 budget；核对 `HINDSIGHT_API_REFLECT_WALL_TIMEOUT`
worker 重启后任务不恢复	检查 `HINDSIGHT_API_WORKER_ID` 是否稳定（9.3 节 Helm 与 StatefulSet 说明）
观察长期不更新或整合卡住	`hindsight bank stats` 观察；必要时 `bank consolidation-recover`（4.2 节）
心智模型刷新失败	看 `last_refresh_failed_at` 与（开启 keep_trace 后的）轨迹；用 dry-run 预演；修复后手动刷新一次以恢复自动触发（7.3 节）
MCP 客户端连不上	确认路径 `http://<host>:8888/mcp/{bank_id}/`；用 `get_version().features.mcp` 确认端点已启用（3.2 节）
LLM 报不支持的结构化输出	换用支持结构化输出的模型（官方推荐 Groq 与对应开源模型；模型清单见 models 页）
内存 / 镜像占用过高	全量改 slim；重排改外部服务（12.1 节）
本地模型下载缓慢	设置 `HF_ENDPOINT=https://hf-mirror.com`（2.6 节，此为 Hugging Face 工具链变量）
retain 返回 HTTP 422	Memory Defense 为 block 模式且全部条目命中；改为 redact 或调整规则（11.1 节）
查询报超长错误	recall 查询上限 500 token（6.2 节），精简或拆分查询
CLI 行为与本文不符	CLI 与服务版本可能不匹配；`hindsight --version` 对照官方变更日志（10.4 节）

结束语

本教程从智能体的记忆问题出发，走完了部署、三操作、数据模型、检索机制、派生知识、各类集成、生产部署、运维、安全与调优，最终落到三类可复用的系统设计。Hindsight 当前处于快速演进期（以官方 Changelog 为准），建议把官方文档、Cookbook 与基准页面作为长期参照：文档回答「怎么用」，Cookbook 回答「别人怎么用」，基准页面回答「为什么这样选值得」。完成本教程的读者，已具备把记忆能力引入现有智能体体系并长期运营的完整基础。